

Practical 14

React Frontend with Spring Boot Backend

Integrate React frontend with Spring Boot API

Aim

To develop a simple Bank Management System using ReactJS and Spring Boot that performs account creation and displays account details using REST APIs and React state management.

Learning Objectives

1. To understand the basics of ReactJS components and state management.
2. To learn how to use React Hooks such as useState.
3. To understand how to create REST APIs using Spring Boot.
4. To learn CRUD operations using Spring Data JPA.
5. To understand frontend and backend integration using Axios.
6. To learn how data is transferred using props and JSON.
7. To understand database connectivity using JPA and MySQL.
8. To learn how to handle form inputs in ReactJS.
9. To understand asynchronous API calls using Axios.
10. To display dynamic data fetched from the backend in ReactJS.

Software Requirements

Hardware Requirements

- Processor: Intel i3 or higher
- RAM: Minimum 4 GB
- Hard Disk: 20 GB free space

Software Requirements

- Operating System: Windows/Linux/Mac
- Java JDK 17 or above
- Spring Boot
- ReactJS
- Node.js and npm
- MySQL Database
- IntelliJ IDEA / Eclipse / VS Code
- Maven
- Web Browser (Chrome/Firefox)
- Axios Library

Input:

AccountController.java

```
package com.shriyash.mca.controller;
```

Python Programming Lab – MCA Sem II

```
import com.shriyash.mca.entity.Account;
import com.shriyash.mca.repository.AccountRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
```

```
import java.util.List;
```

```
@RestController
@RequestMapping("/bank")
@CrossOrigin(origins = "http://localhost:3000")
public class AccountController {

    @Autowired
    private AccountRepository accountRepository;

    @GetMapping("/show")
    public List<Account> getAllAccounts() {
        return accountRepository.findAll();
    }

    @PostMapping("/create")
    public Account createAccount(@RequestBody Account account) {
        return accountRepository.save(account);
    }
}
```

Account.java

```
package com.shriyash.mca.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "accounts")
public class Account {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private Double balance;
```

```
public Account() {}

public Account(String name, Double balance) {
    this.name = name;
    this.balance = balance;
}

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public Double getBalance() {
    return balance;
}

public void setBalance(Double balance) {
    this.balance = balance;
}
}
```

AccountRepository.java

```
package com.shriyash.mca.repository;

import com.shriyash.mca.entity.Account;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface AccountRepository extends JpaRepository<Account, Long> {
}
```

ReactJS

App.js

```
import { useState } from "react";
import axios from "axios";

function App() {
  const [accounts, setAccounts] = useState([]);
  const [showAccounts, setShowAccounts] = useState(false);
  const [name, setName] = useState("");
  const [balance, setBalance] = useState("");

  // View Accounts
  const loadAccounts = async () => {
    try {
      const response = await axios.get("http://localhost:8082/bank/show");
      setAccounts(response.data);
    } catch (error) {
      console.log(error);
    }
  };

  // Create Account
  const createAccount = async (e) => {
    e.preventDefault();
    try {
      await axios.post("http://localhost:8082/bank/create", {
        name: name,
        balance: balance
      });
      alert("Account Created Successfully");
      setName("");
      setBalance("");
    } catch (error) {
      console.log(error);
    }
  };

  return (
    <div style={{ padding: "20px" }}>
      <h1>Bank Management System</h1>
      <h2>Create Account</h2>
      <form onSubmit={createAccount}>
        <input
```

```
    type="text"
    placeholder="Enter Name"
    value={name}
    onChange={(e) => setName(e.target.value)}
  />
  <br /><br />
  <input
    type="number"
    placeholder="Enter Balance"
    value={balance}
    onChange={(e) => setBalance(e.target.value)}
  />
  <br /><br />
  <button type="submit">Create Account</button>
</form>
<hr />
{/* View Accounts */}
<button
  onClick={() => {
    loadAccounts();
    setShowAccounts(true);
  }}
>
  View All Accounts
</button>
<button
  onClick={() => setShowAccounts(false)}
  style={{ marginLeft: "10px" }}
>
  Minimize Accounts
</button>
<br /><br />
{/* Display Accounts */}
{showAccounts &&
  accounts.map((acc) => (
    <div
      key={acc.id}
      style={{
        border: "1px solid black",
        padding: "10px",
        marginBottom: "10px"
      }}
    >
      <h3>{acc.name}</h3>
```

Python Programming Lab – MCA Sem II

```
        <p>ID: {acc.id}</p>
        <p>Balance: {acc.balance}</p>
    </div>
    )}
</div>
);
}
```

export default App;

Output:

